

```

import random

import time

def partition_deterministic(arr, low, high):
    """Deterministic partition: uses the last element as pivot."""
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort_deterministic(arr, low, high):
    """Deterministic Quick Sort."""
    if low < high:
        pi = partition_deterministic(arr, low, high)
        quick_sort_deterministic(arr, low, pi - 1)
        quick_sort_deterministic(arr, pi + 1, high)

def partition_randomized(arr, low, high):
    """Randomized partition: chooses a random element as pivot."""
    random_index = random.randint(low, high)
    arr[random_index], arr[high] = arr[high], arr[random_index] # Swap random pivot to end
    return partition_deterministic(arr, low, high)

def quick_sort_randomized(arr, low, high):
    """Randomized Quick Sort."""
    if low < high:
        pi = partition_randomized(arr, low, high)
        quick_sort_randomized(arr, low, pi - 1)

```

```
quick_sort_randomized(arr, pi + 1, high)
```

```
def analyze_quick_sort(sort_function, arr_size, num_trials=10):
```

```
    """Analyzes the performance of a given Quick Sort variant."""
```

```
    total_time = 0
```

```
    for _ in range(num_trials):
```

```
        data = [random.randint(0, arr_size * 10) for _ in range(arr_size)]
```

```
        arr_copy = list(data) # Create a copy to avoid modifying original
```

```
        start_time = time.perf_counter_ns()
```

```
        sort_function(arr_copy, 0, len(arr_copy) - 1)
```

```
        end_time = time.perf_counter_ns()
```

```
        total_time += (end_time - start_time)
```

```
    avg_time_ms = (total_time / num_trials) / 1_000_000
```

```
    return avg_time_ms
```

```
if __name__ == "__main__":
```

```
    array_sizes = [1000, 5000, 10000]
```

```
    trials = 5
```

```
    print("--- Analysis of Quick Sort Variants ---")
```

```
    for size in array_sizes:
```

```
        print(f"\nArray Size: {size}")
```

```
        # Deterministic Quick Sort
```

```
        avg_time_det = analyze_quick_sort(quick_sort_deterministic, size, trials)
```

```
        print(f"Deterministic Quick Sort (avg over {trials} trials): {avg_time_det:.4f} ms")
```

```
        # Randomized Quick Sort
```

```
        avg_time_rand = analyze_quick_sort(quick_sort_randomized, size, trials)
```

```
        print(f"Randomized Quick Sort (avg over {trials} trials): {avg_time_rand:.4f} ms")
```

```

38 v for _ in range(num_trials):
39     data = [random.randint(0, arr_size * 10) for _ in range(arr_size)]
40     arr_copy = list(data) # Create a copy to avoid modifying original
41     start_time = time.perf_counter_ns()
42     sort_function(arr_copy, 0, len(arr_copy) - 1)
43     end_time = time.perf_counter_ns()
44     total_time += (end_time - start_time)
45     avg_time_ms = (total_time / num_trials) / 1_000_000
46     return avg_time_ms
47
48 v if __name__ == "__main__":
49     array_sizes = [1000, 5000, 10000]
50     trials = 5
51
52     print("--- Analysis of Quick Sort Variants ---")
53
54 v for size in array_sizes:
55     print(f"\nArray Size: {size}")
56
57     # Deterministic Quick Sort
58     avg_time_det = analyze_quick_sort(quick_sort_deterministic, size, trials)
59     print(f"Deterministic Quick Sort (avg over {trials} trials): {avg_time_det:4f} ms")
60
61     # Randomized Quick Sort
62     avg_time_rand = analyze_quick_sort(quick_sort_randomized, size, trials)
63     print(f"Randomized Quick Sort (avg over {trials} trials): {avg_time_rand:4f} ms")

```

--- Analysis of Quick Sort Variants ---

Array Size: 1000

Deterministic Quick Sort (avg over 5 trials): 3.0200 ms

Randomized Quick Sort (avg over 5 trials): 3.0000 ms

Array Size: 5000

Deterministic Quick Sort (avg over 5 trials): 10.9400 ms

Randomized Quick Sort (avg over 5 trials): 16.2600 ms

Array Size: 10000

Deterministic Quick Sort (avg over 5 trials): 24.5200 ms

Randomized Quick Sort (avg over 5 trials): 35.4400 ms